

Name: Hadijah Kyampeire

Registration #: 618990

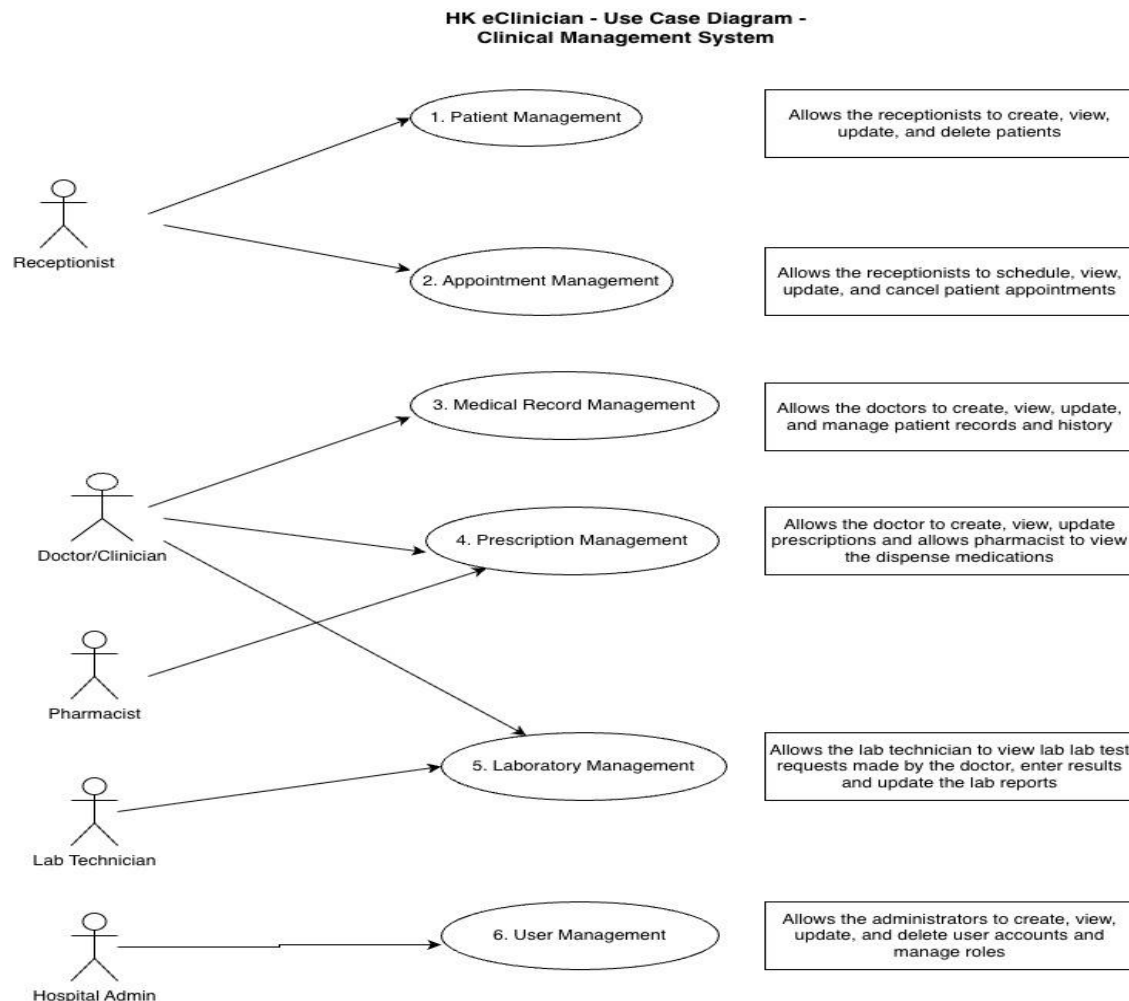
GITHUB LINK: <https://github.com/hadijahkyampeire/eClinician>

eClinician Management System/Software Requirements Specifications (SRS)

The **eClinician Clinical Management System** is a web-based application designed to support healthcare facilities in managing patient registration, appointments, medical records, prescriptions, laboratory requests, and user accounts through a centralized platform. The system aims to improve the efficiency and accuracy of clinical operations by reducing manual record keeping and providing secure, role-based access for healthcare personnel.

This **Software/System Requirements Specification (SRS)** presents the system requirements using a **Use-Case Model**, consisting of a Use-Case Diagram and detailed Use-Case Descriptions. These use cases define how the different users of the system interact with the application and serve as a foundation for the subsequent design and implementation phases of the project.

USE-CASE DIAGRAM:



USE-CASE DESCRIPTIONS:

Use Case Number: 1

Name: Patient Management

Brief description: This use case allows the receptionist to create, view, update, and delete patient profiles.

Actors: Receptionist

Preconditions: The receptionist must be logged in to the system.

Flows of Events:

1. Basic Flows

1.0 Create Patient Profile

Step	User Actions	System Actions
1	The receptionist calls the create patient profile command.	The system displays the patient profile form with the fields for firstname, lastname, date of birth, gender, phone number, national ID / passport number, email and address.
2	The receptionist fills out the form and requests the system to save the details.	The system verifies that no other patient exists in the database with the same national ID / passport number or phone number, saves the patient and returns a success message on success or a fail message in case of failure. In case another profile exists with the same identifier, the system returns a message indicating a duplicate entry exists.

Postconditions:

The patient profile is persisted in the system.

Business Rules:

No duplicate patient profiles. A **unique patient** is identified by patient ID, phone number, or national ID / passport number.

1.1 Read/View Patient Profile

Step	User Actions	System Actions
1	The receptionist selects to view a list of patient profiles.	The system returns a list of all patient profiles.
2	The receptionist selects to view the profile of one of the patients from the list.	The system returns the profile of the patient as a string with the names (firstname and lastname), date of birth, gender, phone number, national ID / passport number, email and address.

1.2 Update Patient Profile

Step	User Actions	System Actions
1	The receptionist selects to view a list of patient profiles.	The system returns a list of all patient profiles.
2	The receptionist selects the profile of the patient they want to update.	The system displays an editable patient profile form pre-populated with the patient profile details.
3	The receptionist updates the fields they want to update and requests the system to save the new details.	The system updates the record and returns a success message or a fail message on exception.

Postconditions: The patient profile will be updated.

Business Rules: The national ID / passport number field should be unwritable.

1.3 Delete Patient Profile

Step	User Actions	System Actions
1	The receptionist selects to view a list of patient profiles.	The system returns a list of all patient profiles.
2	The receptionist selects to delete a patient profile from the list of patient profiles.	The system displays a confirmation dialogue window.
3	The receptionist selects OK on the confirmation dialog window to confirm deleting the profile.	The system confirms the patient is not linked to any appointments, medical records or prescriptions and deletes the profile. The system returns a success message on success or a failure message. The system should return a message indicating the profile could not be deleted because it is linked to existing records.

Postconditions: The patient profile will be deleted.

Business Rules:

Only a patient profile that is not linked to any appointments, medical records or prescriptions can be deleted.

Use Case Number: 2

Name: Appointment Management

Brief description: This use case provides functionality for managing patient appointments.

Actors: Receptionist

Preconditions: The receptionist must be logged in to the system.

Flows of Events:

1. Basic Flows

1.0 Schedule Appointment

Step	User Actions	System Actions
1	The receptionist calls the schedule appointment command.	The system displays the appointment form with the fields for patient, doctor, appointment date, appointment time and reason for visit.
2	The receptionist fills out the form and requests the system to save the details.	The system verifies that the selected doctor does not already have an appointment at the same date and time, and that the patient does not have a duplicate appointment with the same doctor at the same time, then saves the appointment and returns a success message. The system returns a fail message on exception or a conflict message in case of a scheduling clash.

Postconditions: The appointment is persisted in the system.

Business Rules:

A doctor should not have two appointments scheduled at the same date and time.

A patient should not have duplicate appointments for the same doctor and time.

1.1 View Appointment

Step	User Actions	System Actions
1	The receptionist requests a list of appointments.	The system returns a list of all appointments.
2	The receptionist selects to view an appointment they want from the list.	The system returns the appointment object as a string with the patient name, doctor name, appointment date, appointment time and status.

1.2 Update Appointment

Step	User Actions	System Actions
1	The receptionist requests a list of appointments.	The system returns a list of all appointments.
2	The receptionist selects the appointment they want to update.	The system displays an editable appointment form pre-populated with the appointment details.
3	The receptionist changes the parts they want to update and requests the system to save the new details.	The system re-checks for scheduling conflicts, updates the appointment and returns a success message or a fail message on exception.

Postconditions:

The appointment details will be updated.

1.3 Cancel Appointment

Step	User Actions	System Actions
1	The receptionist requests a list of appointments.	The system returns a list of all appointments.
2	The receptionist selects to cancel an appointment from the list.	The system displays a confirmation dialogue window.
3	The receptionist selects OK on the confirmation dialog window to confirm cancelling the appointment.	The system marks the appointment as cancelled and returns a success message or a fail message on exception.

Postconditions: The appointment will be cancelled.

Business Rules: An appointment that has already taken place cannot be cancelled.

Use Case Number: 3

Name: Medical Record Management

Brief description: This use case provides functionality for managing patient consultation records.

Actors: Doctor / Clinician

Preconditions: The clinician must be logged in to the system.

Flows of Events:

1. Basic Flows

1.0 Create Consultation Record

Step	User Actions	System Actions
1	The clinician calls the create consultation record command.	The system displays the consultation record form with the fields for patient, symptoms, diagnosis, notes and consultation date.
2	The clinician fills out the form and requests the system to save the details.	The system verifies that the record is linked to an existing patient, saves the consultation record and returns a success message or a fail message on exception.

Postconditions: The consultation record is persisted in the system.

Business Rules: Only clinicians can create or update medical records.

Medical records should be linked to an existing patient.

1.1 View Patient Medical History

Step	User Actions	System Actions
1	The clinician selects a patient and requests their medical history.	The system returns a list of all consultation records linked to the selected patient.
2	The clinician selects a consultation record they want to view from the list.	The system returns the consultation record as a string with the patient name, symptoms, diagnosis, notes and consultation date.

1.2 Update Consultation Record

Step	User Actions	System Actions
1	The clinician selects a patient and requests their medical history.	The system returns a list of all consultation records linked to the patient.
2	The clinician selects the consultation record they want to update.	The system displays an editable consultation record form pre-populated with the record details.
3	The clinician changes the parts they want to update and requests the system to save the new details.	The system updates the record and returns a success message or a fail message on exception.

Postconditions: The consultation record will be updated.

Business Rules: A consultation record can only be updated by a clinician.

Use Case Number: 4

Name: Prescription Management

Brief description: This use case allows a doctor to issue prescriptions and a pharmacist to view and dispense the prescribed medication.

Actors: Doctor, Pharmacist

Preconditions: The doctor or pharmacist must be logged in to the system.

Flows of Events:

1. Basic Flows

1.0 Create Prescription

Step	User Actions	System Actions
1	The doctor calls the create prescription command.	The system displays the prescription form with the fields for patient, consultation, medication, dosage, frequency and duration.
2	The doctor fills out the form and requests the system to save the details.	The system verifies that the prescription is linked to an existing patient and consultation, saves the prescription and returns a success message or a fail message on exception.

Postconditions: The prescription is persisted in the system.

Business Rules: Only doctors can issue prescriptions.

A prescription must be linked to a patient and a consultation.

1.1 View Prescription

Step	User Actions	System Actions
1	The doctor or pharmacist requests a list of prescriptions.	The system returns a list of all prescriptions.

Step	User Actions	System Actions
2	The doctor or pharmacist selects a prescription they want to view from the list.	The system returns the prescription object as a string with the patient name, medication, dosage, frequency, duration and dispensed status.

1.2 Dispense Medication

Step	User Actions	System Actions
1	The pharmacist requests a list of prescriptions.	The system returns a list of all prescriptions that are not yet dispensed.
2	The pharmacist selects a prescription and marks the medication as dispensed.	The system updates the prescription status to dispensed and returns a success message or a fail message on exception.

Postconditions: The prescription will be marked as dispensed.

Business Rules: Only pharmacists can mark medication as dispensed.

Use Case Number: 5

Name: Laboratory Management

Brief description: This use case allows a doctor to request laboratory tests and a laboratory technician to record the results.

Actors: Doctor, Laboratory Technician

Preconditions: The doctor or laboratory technician must be logged in to the system.

Flows of Events:

1. Basic Flows

1.0 Create Laboratory Request

Step	User Actions	System Actions
1	The doctor calls the create laboratory request command.	The system displays the laboratory request form with the fields for patient, consultation, test type and notes.
2	The doctor fills out the form and requests the system to save the details.	The system verifies that the request is linked to an existing patient, saves the laboratory request and returns a success message or a fail message on exception.

Postconditions: The laboratory request is persisted in the system.

Business Rules: A laboratory request must be linked to an existing patient.

1.1 View Laboratory Request

Step	User Actions	System Actions
1	The doctor or laboratory technician requests a list of laboratory requests.	The system returns a list of all laboratory requests.

Step	User Actions	System Actions
2	The user selects a laboratory request they want to view from the list.	The system returns the laboratory request object as a string with the patient name, test type, notes and status.

1.2 Record Laboratory Results

Step	User Actions	System Actions
1	The laboratory technician requests a list of pending laboratory requests.	The system returns a list of all laboratory requests that do not yet have results.
2	The laboratory technician selects a request and enters the results, then requests the system to save.	The system verifies the results are linked to an existing laboratory request, saves the results and returns a success message or a fail message on exception.

Postconditions: The laboratory results are persisted in the system.

Business Rules: Lab results must be linked to a lab request.

Only laboratory technicians can record laboratory results.

1.3 View Laboratory Results

Step	User Actions	System Actions
1	The doctor selects a patient and requests their laboratory results.	The system returns a list of all laboratory results linked to the patient.
2	The doctor selects a laboratory result they want to view from the list.	The system returns the laboratory result as a string with the patient name, test type, results and result date.

Business Rules: Doctors can view laboratory results for their patients.

Use Case Number: 6

Name: User Management

Brief description: This use case provides functionality for managing system users and their roles.

Actors: Administrator

Preconditions: The administrator must be logged in to the system.

Flows of Events:

1. Basic Flows

1.0 Create User

Step	User Actions	System Actions
1	The administrator calls the create user command.	The system displays the user form with the fields for firstname, lastname, email / username, password and role.
2	The administrator fills out the form and requests the system to save the details.	The system verifies that no other user exists with the same email / username, saves the

Step	User Actions	System Actions
		user and returns a success message. The system returns a fail message on exception or a duplicate entry message in case of a duplicate.

Postconditions: The user account is persisted in the system.

Business Rules: No duplicate user accounts. - Email / username must be unique.

1.1 View User

Step	User Actions	System Actions
1	The administrator requests a list of users.	The system returns a list of all users.
2	The administrator selects a user they want to view from the list.	The system returns the user object as a string with the names (firstname, lastname), email / username and role.

1.2 Update User

Step	User Actions	System Actions
1	The administrator requests a list of users.	The system returns a list of all users.
2	The administrator selects the user they want to update.	The system displays an editable user form pre-populated with the user details.
3	The administrator changes the parts they want to update and requests the system to save the new details.	The system updates the record and returns a success message or a fail message on exception.

Postconditions: The user details will be updated.

Business Rules: The email / username field should be unwritable.

1.3 Delete / Deactivate User

Step	User Actions	System Actions
1	The administrator requests a list of users.	The system returns a list of all users.
2	The administrator selects to delete or deactivate a user from the list.	The system displays a confirmation dialogue window.
3	The administrator selects OK on the confirmation dialog window to confirm the action.	The system deactivates or deletes the user account and returns a success message or a fail message on exception.

Postconditions: The user account will be deleted or deactivated.

Business Rules: Users should only access features allowed by their role.